目录 (Index)

- ez_uds
- Illusion
- hajimi
- 移动的秘密
- Hulua
- ezFinger
- Disguise
- ezLanguage

---

ez_uds

看算法很简单，就是随机数生成一个seed，然后seed经过一个算法异或移位啥的求出key，key发到容器获得flag。 再看uds的协议，得知27是安全访问，01申请seed，02发key。 所以写一个交互脚本

```
import socket
import time

def calculate_key(seed):
    key = seed ^ 0xA5A5A5A5
    key = ((key << 3) | (key >> 29)) & 0xFFFFFFFF
    key = (key + 0x12345678) & 0xFFFFFFFF
    return key

s = socket.socket()
s.connect(("nc1.ctfplus.cn", 36105))
time.sleep(1)
s.recv(4096)

s.sendall(b"27 01\n")
time.sleep(1)

res = s.recv(1024).decode()
a = res.split()
seed = int(a[2] + a[3] + a[4] + a[5], 16)

key = calculate_key(seed)

k = hex(key)[2:].zfill(8).upper()
ans = "27 02 " + k[0:2] + " " + k[2:4] + " " + k[4:6] + " " + k[6:8] + "\n"

s.sendall(ans.encode())
time.sleep(1)
```

```
print(s.recv(4096).decode())
```

得到flag

Illusion

拖进ida中看main函数逻辑。 读取输入不超过25字节。然后检查格式是xmctf{}。 中间再检查一下长度。 然后就是给出一个密文记得小端绪，出来是0xD5, 0x0A, 0xFB, 0x84, 0x0A, 0x8F, 0x2C, 0xE7, 0x27, 0xD9, 0x56, 0x3E, 0xF3, 0x6C, 0x29, 0xAB, 0x19, 0x54。 接下来应该就是这个18字节的密文sub_140001440经过这个函数与输入的{}中的18个字节做比较的。 再深入看这个sub_140001440，说实话加密算法都不想分析，让ai看出是经典rc4，密钥是nev_gona_give_up。厨子里出来nev_gona_letydown。 得到flag是xmctf{nev_gona_letydown}。 靠，发现是假的 看字符串发现有个real world 跟过去找到sub_140001000，是个tls。hook了MessageBoxA api改为了跳转进真正的主逻辑。 这里有个反调试不过没关系，返回去看终于弄懂了，输入假的flag不会触发弹窗api，进入不了这个主逻辑。 sub_1400010F0才是真正的主逻辑，是个aes，解密 找到密钥123412341234123412341234AES!，密文。 得出真正flag xmctf{R3a1_w0rld_M47ters}

hajimi

看不懂，搜了一下导入的模块，好像是和神经网络深度学习有关。 让ai解释了一下，逻辑感觉就是输入一个只有1到4的16字节字符串进入了一个黑盒。那应该就是要逆challenge.pkl.zst这个东西。但我又不会，看着好麻烦，尝试爆破吧，ai提示1到4 16个是给4乘4数独（我倒是没看出来），反正尝试爆破最优。 补好环境，写个爆破脚本，塞进模型，得出 1234341221434321。 然后得出flag是xmctf{b0a0d1edc0fb5b75770a5dcbe7b0d4fb08e42fd281a94ee67b405e36056f1df1}。 （这题思路基本都是ai给） 总和起来的脚本

```
import itertools
import sys
import importlib.util
import os
import hashlib

os.environ['JAX_PLATFORMS'] = 'cpu'

spec = importlib.util.spec_from_file_location('main_mod', '__main__.py')
main_mod = importlib.util.module_from_spec(spec)
sys.modules['main_mod'] = main_mod
spec.loader.exec_module(main_mod)

def check_grid(s):
    t = [list(s[i:i+4]) for i in range(0, 16, 4)]
    for r in t:
        if len(set(r)) != 4: return False
    for c in range(4):
```

```
        if len(set([t[i][c] for i in range(4)])) != 4: return False
    for i in range(0, 4, 2):
        for j in range(0, 4, 2):
            box = [t[i][j], t[i][j+1], t[i+1][j], t[i+1][j+1]]
            if len(set(box)) != 4: return False
    return True

all_perm = list(itertools.permutations('1234'))
good_ones = []

for r1 in all_perm:
    for r2 in all_perm:
        for r3 in all_perm:
            for r4 in all_perm:
                s = "".join(r1+r2+r3+r4)
                if check_grid(s):
                    good_ones.append(s)

my_model = main_mod.load_model('challenge.pkl.zst')

for g in good_ones:
    ans = main_mod.decode_output(my_model.apply(['BOS'] + list(g)))
    if "wrong" not in ans.lower() and "grid" not in ans.lower() or len(ans) > 5:
        h = hashlib.sha256(g.encode('utf8')).hexdigest()
        print("xmctf{" + h + "}")
        break
```

移动的秘密

进入main函数，有个ptrace作反调试，不过这题应该不用动调。 继续看要求目标输入29字节字符串，然后开始遍历右移。这里v14应该是3080前13个加3090组成的预期移位后的结果，[60, 54, 49, 58, 51, 61, 59, 50, 54, 49, 24, 54, 50, 47, 25, 47, 56, 55, 54, 48, 57, 24, 57, 47, 24, 24, 25, 25, 62]这样 但这样移位会导致缺少一位。 再看下面应该还会多一层检验，懒得看算法，ai看出来是哈希检验，3060应该是它的魔数。3070是哈希值作为预期结果。 现在理清可以写爆破脚本了

```
import hashlib
import itertools

def solve():
    part1 = [60, 54, 49, 58, 51, 61, 59, 50, 54, 49, 24, 54, 50]
    part2 = [47, 25, 47, 56, 55, 54, 48, 57, 24, 57, 47, 24, 24, 25, 25, 62]
    targets = part1 + part2

    target_md5 = "3a22c098710019b31c328a861429d3ad"
```

```
    possible_chars = []
    for t in targets:
        c1 = t * 2
        c2 = t * 2 + 1

        valid_chars = []
        if 32 <= c1 <= 126:
            valid_chars.append(chr(c1))
        if 32 <= c2 <= 126:
            valid_chars.append(chr(c2))

        possible_chars.append(valid_chars)

    count = 0

    for combo in itertools.product(*possible_chars):
        flag = "".join(combo)

        h = hashlib.md5(flag.encode()).hexdigest()

        if h == target_md5:
            print("find flag: " + flag)
            return

        count += 1
        if count % 10000000 == 0:
            print("counting...", count)

if __name__ == '__main__':
    solve()
```

得出flag xmctf{welc0me_2_polar1s_1022}

Hulua

字符串引用找到main在sub_14000157F，看一下,依旧没咋看懂。 看题目中有lua，字符串里还看到lua的版本，应该是之前听过的lua逆向。上网搜了两篇文章，大概弄懂了。应该是c中内嵌了lua，主逻辑先获取用户输入，然后创建v4作为lua state，并将输入赋给全局变量 user_input,从而顺利传给lua。那这题目的就是找到真正的主逻辑，也就是lua脚本，那就应该dump出lua字节码luac文件。继续分析伪代码，`v3 = sub_140016310(v4, &dword_140033000, dword_1400333DC, "check", 0);` 这个调用的是luaL_loadbuffer这个lua api，那就能知道lua脚本就是0x140033000从这开始的一大串，发现还被sub_1400014A4函数引用，跟过去发现被异或加密了一下。 写个idapython脚本提取一下

```
import idc
```

```
def dump_payload():
    start_addr = 0x140033000
    size_addr = 0x1400333DC
    key = b"hulua"

    script_size = idc.get_wide_dword(size_addr)
    raw_data = idc.get_bytes(start_addr, script_size)

    if not raw_data:
        print("[-] Failed to read bytes.")
        return

    out_data = bytearray()
    for i in range(script_size):
        out_data.append(raw_data[i] ^ key[i % len(key)])

    out_path = r"D:\challenge\6\check_decrypted.luac"
    with open(out_path, "wb") as f:
        f.write(out_data)

    print(f"[+] Dumped to {out_path}")
    print(f"[+] Header: {out_data[:4]}")

dump_payload()
```

出来之后再异或解密一下得到luac文件，用unluac一键反编译报错了，看信息好像是加了混淆，问了下ai只能用unluac --disassemble提取一下常量来看看，发现两行特殊的hex，一个是"78 6D 63 74 66 32 30 32 36"转为字符串是xmctf2026感觉应该是key之类的，另一行"8B 8B 77 BE 68 61 86 68 E5 63 EE 84 35 6F 58 C8 51 0F 6E 94 70 E7 26 90 B6 75 EC 28 AF 14 E2 E3"转不出来，推测是密文吧。再看下面有256和255感觉应该是个rc4，然后还有个102不知道什么意思，猜着可能是异或，放在厨子里一算xmctf{lu4t1c_r3v3rs3_ch4ll3ng3!}，猜对了，就没再分析。

ezFinger

这题就是纯ai，没接触过太多。finger插件也分析不出来。感觉思路应该就是去查STM32F429ZI的芯片手册啥的，再去找那些标准库中对应的函数?

Disguise

拖进ida看字符串发现主逻辑不在那，而且还存在一堆没用的花指令和堆栈不平衡。再看导入表也没有几个api，应该是个壳。看到VirtualAlloc应该就是壳运行后将主程序解密用它来申请内存空间，所以追踪这个api，动调并dump出内存中的主程序好了。 放到xdbg里面，发现f9两下直接不动了。应该是有啥反调试，直接scyllahide插件一把梭，去了反调试继续追踪,先bp VirtualAlloc下个断点，然后执行完这个api，找到分配内存的地址。跟过去下个硬件断点，运行完这个api，发现该处内存出现了pe头，最后把这个主程序dump出来。得到一个bin文件，还不能直接丢到ida里，先修复一下pe结构。 最后终于得到了脱壳后的程序，字符

串引用找到主逻辑。先要输入一个48字节的字符串，看到sub_411401应该是一个加密函数跟进去，发现是一个vm，再看sub_4154A0应该就是一个分发器。有点复杂，直接丢给ai看出这里实现的是魔改的sm4。写个脚本总结下结束

```
import pefile
import struct

def read_dwords(data, offset, count):
    return list(struct.unpack(f'<{count}I', data[offset:offset+count*4]))

def read_bytes(data, offset, count):
    return list(data[offset:offset+count])

def disassemble_vm(data, base_addr, text_offset=0):
    vm_code_offset = 0x422020 - base_addr
    words = read_dwords(data, vm_code_offset, 2500)

    print("-" * 50)
    print("[+] 正在基于 Opcode 分析并输出前 30 条 VM 指令反汇编...")
    print("-" * 50)

    pc = 0
    instructions_decoded = 0
    while pc < len(words) and instructions_decoded < 30:
        opcode = words[pc]
        op_name = f"OP_{opcode}"
        length = 1
        disasm = ""

        if opcode == 0:
            disasm = "VM_INIT ()"
            length = 1
        elif opcode == 1:
            r1, imm = words[pc+1], words[pc+2]
            disasm = f"ADD R{r1}, {hex(imm)}"
            length = 3
        elif opcode == 2:
            r1, r2 = words[pc+1], words[pc+2]
            disasm = f"XOR R{r1}, R{r2}"
            length = 3
        elif opcode == 3:
            r1, r2 = words[pc+1], words[pc+2]
            disasm = f"SUB R{r1}, R{r2}"
            length = 3
        elif opcode == 4:
            r1, r2 = words[pc+1], words[pc+2]
```

```
        disasm = f"MOV R{r1}, R{r2}"
        length = 3
    elif opcode == 6:
        r1, imm = words[pc+1], words[pc+2]
        disasm = f"SHL R{r1}, {hex(imm)}"
        length = 3
    elif opcode == 7:
        r1, imm = words[pc+1], words[pc+2]
        disasm = f"SHR R{r1}, {hex(imm)}"
        length = 3
    elif opcode == 9:
        disasm = "JNZ TARGET"
        length = 1
    elif opcode == 10:
        disasm = "JZ TARGET"
        length = 1
    elif opcode == 11:
        r1, imm = words[pc+1], words[pc+2]
        disasm = f"CMP R{r1}, {hex(imm)}"
        length = 3
    elif opcode == 14:
        r1, imm = words[pc+1], words[pc+2]
        disasm = f"AND R{r1}, {hex(imm)}"
        length = 3
    elif opcode == 15:
        r1, r2 = words[pc+1], words[pc+2]
        disasm = f"OR R{r1}, R{r2}"
        length = 3
    elif opcode == 16:
        disasm = "VM_EXIT ()"
        length = 1
    elif opcode == 17:
        r1, r2, r3 = words[pc+1], words[pc+2], words[pc+3]
        disasm = f"LDR R{r1}, [R{r2} + R{r3}*4]"
        length = 4
    elif opcode == 18:
        r1, r2, r3 = words[pc+1], words[pc+2], words[pc+3]
        disasm = f"STR [R{r1} + R{r2}*4], R{r3}"
        length = 4
    elif opcode == 19:
        disasm = "NOP"
        length = 2
    elif opcode == 26:
        disasm = "SET_TARGET"
        length = 1
    else:
```

```
            disasm = f"UNKNOWN ({opcode})"
            break

        print(f"[{pc:04d}] {op_name:<10} : {disasm}")
        pc += length
        instructions_decoded += 1

    print("-" * 50)
    print("[+] VM 指令解析可知：大量 SHL, SHR, XOR 是典型的密码学移位操作组合。")


def decrypt_sm4(data, base_addr):
    SBOX = read_dwords(data, 0x41ec30 - base_addr, 256)
    FK_CONST = read_dwords(data, 0x422010 - base_addr, 4)
    CK_CONST = read_dwords(data, 0x41f030 - base_addr, 32)
    CT = read_dwords(data, 0x421018 - base_addr, 12)

    print(f"\n[+] 成功提取 SBox (前4个): {SBOX[:4]} ...")
    print(f"[+] 成功提取 FK: {[hex(x) for x in FK_CONST]}")
    print(f"[+] 成功提取 目标密文 CT: {[hex(x) for x in CT]}")

    def L_prime(b):
        return b ^ ((b << 13) | (b >> 19)) & 0xFFFFFFFF ^ ((b << 23) | (b >> 9)) &
0xFFFFFFFF

    def L(b):
        return b ^ ((b << 2) | (b >> 30)) & 0xFFFFFFFF ^ ((b << 10) | (b >> 22)) &
0xFFFFFFFF ^ \
               ((b << 18) | (b >> 14)) & 0xFFFFFFFF ^ ((b << 24) | (b >> 8)) & 0xFFFFFFFF

    def T_prime(val):
        b0, b1, b2, b3 = SBOX[(val >> 24) & 0xFF], SBOX[(val >> 16) & 0xFF], SBOX[(val >>
8) & 0xFF], SBOX[val & 0xFF]
        return L_prime((b0 << 24) | (b1 << 16) | (b2 << 8) | b3)

    def T(val):
        b0, b1, b2, b3 = SBOX[(val >> 24) & 0xFF], SBOX[(val >> 16) & 0xFF], SBOX[(val >>
8) & 0xFF], SBOX[val & 0xFF]
        return L((b0 << 24) | (b1 << 16) | (b2 << 8) | b3)

    def expand_key(key_bytes):
        K = list(struct.unpack('>4I', key_bytes))
        rk = []
        for i in range(4): K[i] ^= FK_CONST[i]
        for i in range(32):
            val = T_prime(K[(i+1)%4] ^ K[(i+2)%4] ^ K[(i+3)%4] ^ CK_CONST[i])
```

```
            K[i%4] ^= val
            rk.append(K[i%4])
        return rk

    def decrypt_block(ct, rk):
        X = list(ct)
        for i in range(32):
            val = T(X[(i+1)%4] ^ X[(i+2)%4] ^ X[(i+3)%4] ^ rk[31-i])
            X[i%4] ^= val
        return struct.pack('>4I', X[3], X[2], X[1], X[0])

    print("\n[+] 正在根据 Custom SM4 定义进行解密运算...")
    rk = expand_key(b'WelcOme_tO_xmctf')
    flag = b''
    for i in range(0, len(CT), 4):
        flag += decrypt_block(CT[i:i+4], rk)

    print("\n" + "=" * 50)
    print(f"解出的 Flag: {flag.decode('utf-8', errors='ignore')}")
    print("=" * 50)


def main():

    try:
        pe = pefile.PE('dumped.exe')
        data = pe.get_memory_mapped_image()
        base_addr = pe.OPTIONAL_HEADER.ImageBase
    except FileNotFoundError:
        print("[!] 错误: 未找到 dumped.exe，请确保脚本在题目的当前目录下执行。")
        return

    disassemble_vm(data, base_addr)
    decrypt_sm4(data, base_addr)

if __name__ == '__main__':
    main()
```

ezLanguage

拖进die一看有壳，32位，在x32dbg中用esp定理找到oep在用Scylla dump脱壳重建iat表。 然后在拖到ida中，发现有一大堆混淆花指令，写个ida python脚本，批量去除一下。

```
import ida_bytes
import idc
```

```
import ida_auto

def clean_junk():
    start_ea = 0x401000
    end_ea = 0x47b000
    ea = start_ea
    while ea < end_ea:
        b1 = ida_bytes.get_byte(ea)
        b2 = ida_bytes.get_byte(ea+1)
        b3 = ida_bytes.get_byte(ea+2)
        if b1 == 0xEB:
            if b2 == 0x01:
                for i in range(3):
                    ida_bytes.patch_byte(ea+i, 0x90)
                ea += 3
                continue
            elif b2 == 0x02:
                for i in range(4):
                    ida_bytes.patch_byte(ea+i, 0x90)
                ea += 4
                continue
        if (b1 == 0xF8 and b2 == 0x73 and b3 == 0x01) or \
           (b1 == 0xF9 and b2 == 0x72 and b3 == 0x01):
            for i in range(4):
                ida_bytes.patch_byte(ea+i, 0x90)
            ea += 4
            continue
        if b1 == 0x33 and b2 == 0xC0 and b3 == 0x74 and ida_bytes.get_byte(ea+3) == 0x01:
            ida_bytes.patch_byte(ea+2, 0x90)
            ida_bytes.patch_byte(ea+3, 0x90)
            ida_bytes.patch_byte(ea+4, 0x90)
            ea += 5
            continue
        if b1 == 0xE8 and ida_bytes.get_dword(ea+1) == 0:
            if ida_bytes.get_byte(ea+5) == 0x83 and \
               ida_bytes.get_byte(ea+6) == 0x04 and \
               ida_bytes.get_byte(ea+7) == 0x24:
                add_val = ida_bytes.get_byte(ea+8)
                if ida_bytes.get_byte(ea+9) == 0xC3:
                    junk_len = add_val - 5
                    for i in range(10 + junk_len):
                        ida_bytes.patch_byte(ea+i, 0x90)
                    ea += (10 + junk_len)
                    continue
        ea += 1
```

```
clean_junk()
ida_bytes.del_items(0x401000, 0, 0x47B000 - 0x401000)
ida_auto.plan_and_wait(0x401000, 0x47B000)
```

然后从字符串引用找到主逻辑sub_40118F 看着逻辑好乱，还有一堆sub_401714，像是vm的分发器，查了一下好像是易语言的引擎吧。继续分析这段伪代码，大体上只能看懂进行一堆运算，最后与<b<@<72-*8oz*6o-o7co-s73515yk5553<w&znz9640bj&j28+-+8xh44密文对比。只能求助ai，看到是把输入的一个字符分成了两个，对应两个表的索引，所以由密文知道输入的应该是28个字符。 字典长度为 34*: `4&ne9h1<y2*$oics-75wk3a0z@6jv8>+bx` 中间的核心算法为对于第 i 个字符 c (i 始于 1):

索引1: `(c / 17.0 + i - 1) % 34.0 + 1` 索引2: `35.0 - ((c % 17.0 + i) % 34.0)` (遇 35 则置 1) 总之就是商和余数分别附加在i上。 最后因为前后字符之间无关可以通过爆破得出flag

```
def solve():
    cipher = "<b<@<72-*8oz*6o-o7co-s73515yk5553<w&znz9640bj&j28+-+8xh44"
    alphabet = "4&ne9h1<y2*$oics-75wk3a0z@6jv8>+bx"
    flag = ""
    for i in range(1, 29):
        target_a = cipher[(i - 1) * 2]
        target_b = cipher[(i - 1) * 2 + 1]
        for c in range(32, 127):
            idx1 = int((c / 17.0 + i - 1) % 34.0) + 1
            idx2 = int(35.0 - ((c % 17 + i) % 34.0))
            if idx2 == 35: idx2 = 1
            if alphabet[idx1 - 1] == target_a and alphabet[idx2 - 1] == target_b:
                flag += chr(c)
                break
    print(flag)

solve()
```